

Model Evaluation and Inference

IN5148: Statistics and Data Science with Applications in Engineering

Alan R. Vazquez
Department of Industrial Engineering

Agenda

1. Residual analysis
2. Inference about individual β 's using t-tests

Load the libraries

Let's import the required libraries into Python together with other relevant libraries.

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 from sklearn.linear_model import LinearRegression
5 import statsmodels.api as sm
```

We will introduce the new library **statsmodels.api** later.

Inference about individual β 's using t-tests

Linear Regression Model

Recall that the linear regression model is a common candidate function for predicting a response:

$$\hat{Y}_i = \hat{f}(\mathbf{X}_i) = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \cdots + \hat{\beta}_p X_{ip}.$$

- Where $i = 1, \dots, n_t$ is the index of the n_t training data.
- $\hat{Y}_i$ is the prediction of the actual value of the response Y_i associated with values of p predictors denoted by $\mathbf{X}_i = (X_{i1}, \dots, X_{ip})$.
- The values $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ are the *coefficients* of the model.

Example 1

Let's consider the “auto.xlsx” dataset.

```
1 auto_data = pd.read_excel('auto.xlsx')
```

Let's assume that we want to train the following model.

$$\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_{i1} + \hat{\beta}_2 X_{i2},$$

where $\hat{Y}_i$ is the predicted mpg, X_{i1} is the weight, and X_{i2} is the acceleration of the i -th car, $i = 1, \dots, 392$.

Here, we will use the entire dataset for training and ignore K -fold cross-validation to illustrate the concepts.

Research question

Do the weight and acceleration have a significant association with the mpg of a car?

The two cultures of statistical models

- **Inference**: develop a model that fits the data well. Then make inferences about the data-generating process based on the structure of such model.
- **Prediction**: Silent about the underlying mechanism generating the data and allow for many predictive algorithms, which only care about accuracy of predictions.

They overlap very often.

The least squares estimates $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ are subject to uncertainty, since they are calculated based on a random sample of data. That is, an *instance* of training data.

Therefore, assessing the amount of the uncertainty in these estimates is important. To this end, we use **hypothesis** tests on individual coefficients.

Hypothesis test

A statistical hypothesis is a statement about the coefficients of a model.

In our case, we are interested in testing:

$$H_0 : \beta_j = 0 \text{ v.s. } H_1 : \beta_j \neq 0 \text{ (Two-tailed Test)}$$

- β_j is the **true (unknown)** coefficient of the predictor.
- Rejecting H_0 (in favor of H_1) implies that the predictor has a significant association with the response.
- Not rejecting H_0 implies that the predictor does not have a significant association with the response.

$$H_0 : \beta_j = 0 \text{ v.s. } H_1 : \beta_j \neq 0 \text{ (Two-tailed Test)}$$

Testing this hypothesis consists of the following steps:

1. Take a random sample (**your training data**).
2. Compute the appropriate test statistic.
3. Reject or fail to reject the null hypothesis based on a computed p-value.

Step 1. Random sample

The random sample is the training data we use to train or fit the model.

```
1 # Dataset with predictors.  
2 auto_X_train = auto_data[['weight', 'acceleration']]  
3  
4 # Dataset with the response only.  
5 auto_Y_train = auto_data['mpg']
```

As mentioned before, we will use the entire dataset for training.

Step 2. Test statistic

The test statistic is

$$t_0 = \frac{\hat{\beta}_j}{\sqrt{\hat{v}_{jj}}}$$

- $\hat{\beta}_j$ is the least squares estimate of the true coefficient β_j .
- $\sqrt{\hat{v}_{jj}}$ is the *standard error* of the estimate $\hat{\beta}_j$ calculated using Python.

Recall that we obtain the least squares estimates ($\hat{\beta}_j$) in Python using:

```
1 # 1. Create the linear regression model
2 LRmodel = LinearRegression()
3
4 # 2. Fit the model.
5 LRmodel.fit(auto_X_train, auto_Y_train)
6
7 # 3. Show estimated coefficients.
8 print("Intercept:", LRmodel.intercept_)
9 print("Coefficients:", LRmodel.coef_)
```

Intercept: 41.095328779604216

Coefficients: [-0.0072931 0.2616504]

Later, we will see how to obtain the *standard error* of the estimate $\hat{\beta}_j$.

$$t_0 = \frac{\hat{\beta}_j}{\sqrt{\hat{v}_{jj}}}$$

- We like this statistic because it follows a well-known distribution.
- If the null hypothesis ($H_0 : \beta_j = 0$) is true, the statistic t_0 follows a ***t* distribution** with $n - p - 1$ degrees of freedom
- Remember that n is the number of observations and p the number of predictors.

t distribution

This distribution is also known as the student's t-distribution.

It was invented by [William Gosset](#) when he worked at the Guinness Brewery in Ireland.

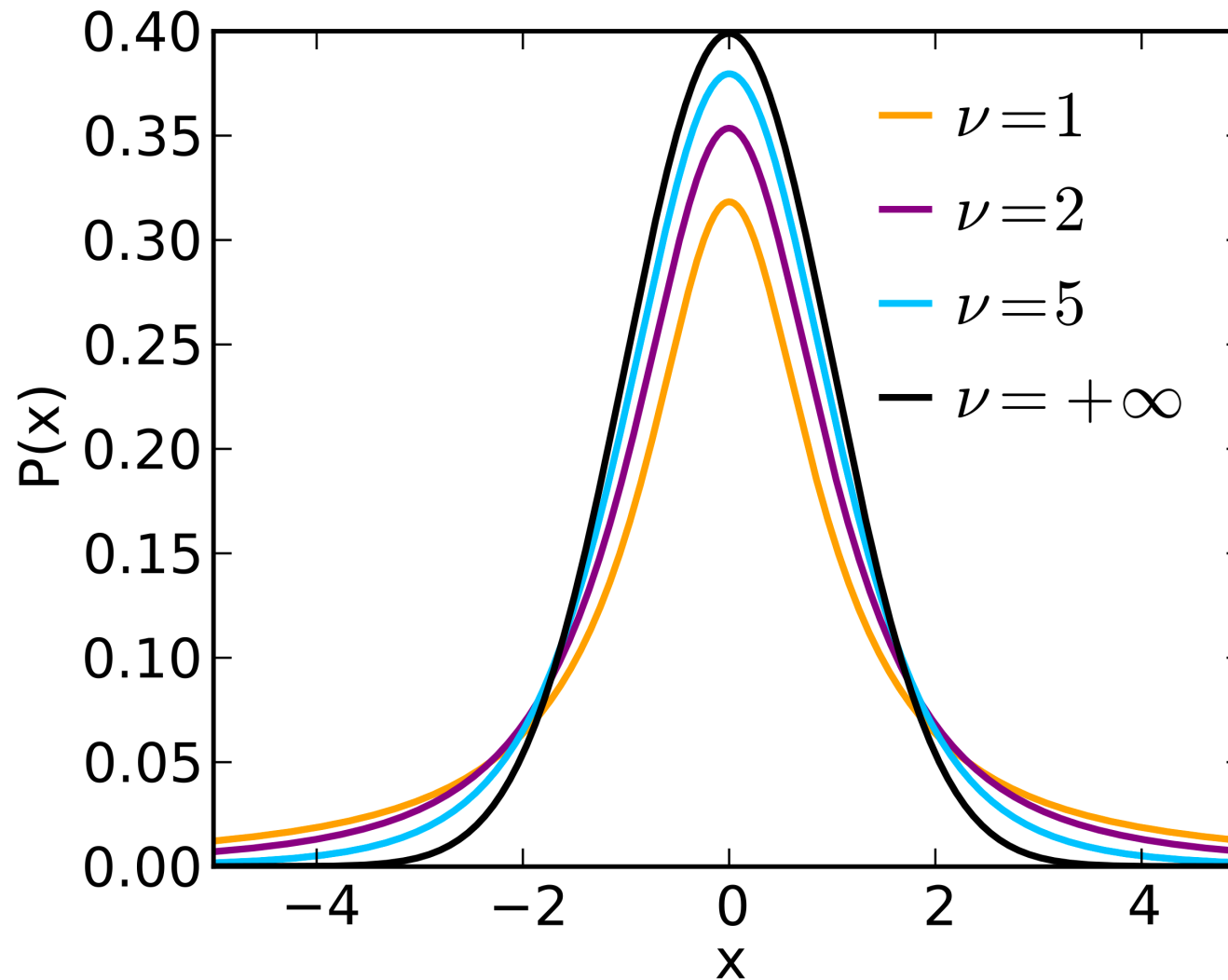
It has one parameter ν which generally equals a number of degrees of freedom.

The parameter ν controls the shape of the distribution.



[William Gosset](#)

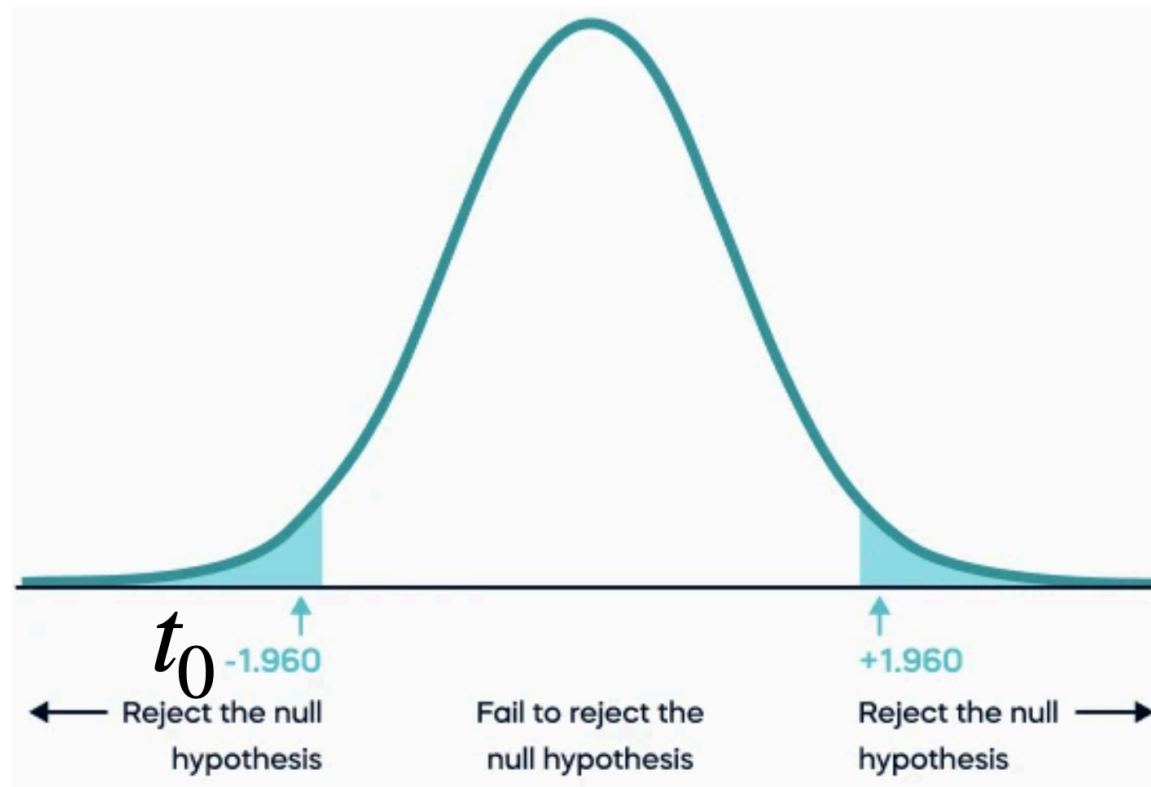
The t-distribution resembles a standard normal distribution when ν goes to infinity.



Step 3. Calculate the p-value

The **p-value** is the probability that the **t test statistic** will get a value that is at least as extreme as the observed t_0 value when the null hypothesis (H_0) is true.

For $H_0 : \beta_j = 0$ v.s. $H_1 : \beta_j \neq 0$, the p-value is calculated using both tails of the t distribution. It is the blue area under the curve below.



We use the two tails because H_1 includes the possibility that the value of β_j is positive or negative.

The p-value is not the probability that H_0 is true

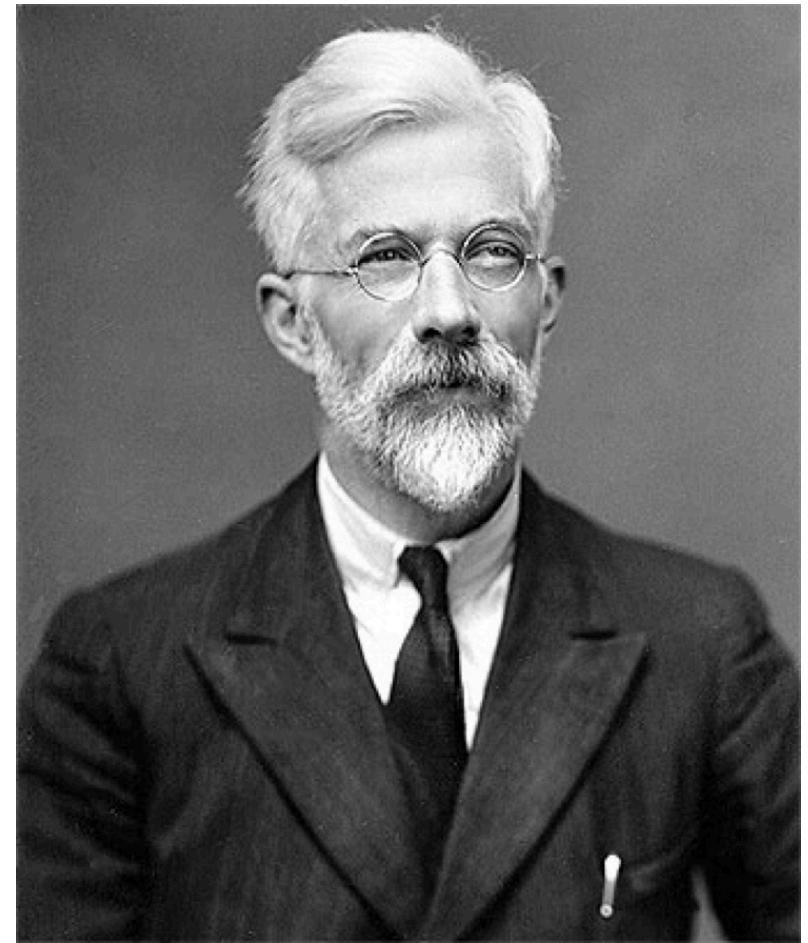
Since the p-value is a probability, and since small p-values indicate that H_0 is unlikely to be true, it is tempting to think that the p-value represents the probability that H_0 is true.

This is not the case!

Remember that the **p-value** is the probability that the t test statistic will get a value that is at least as extreme as the observed t_0 value **when H_0 is true.**

How small must the p-value be to reject H_0 ?

- **Answer:** Very small!
- But, how small?
- **Answer:** Very small!
- But, tell me, how small?
- **Answer:** Okay, it must be less than a value called α which is usually 0.1, 0.05, or 0.01.
- Thank you!



Sir Ronald Fisher

Decision

For a significance level of $\alpha = 0.05$:

- If the p-value is smaller than α , we reject $H_0 : \beta_j = 0$ in favor of $H_1 : \beta_j \neq 0$.
- If the p-value is larger than α , we do not reject $H_0 : \beta_j = 0$.

No scientific basis for this advice. In practice, report the p-value and explore the data using descriptive statistics.

EDITORIAL

The ASA's Statement on p -Values: Context, Process, and Purpose

In February 2014, George Cobb, Professor Emeritus of Mathematics and Statistics at Mount Holyoke College, posed these questions to an ASA discussion forum:

Q: Why do so many colleges and grad schools teach $p = 0.05$?

A: Because that's still what the scientific community and journal editors use.

Q: Why do so many people still use $p = 0.05$?

A: Because that's what they were taught in college or grad school.

Cobb's concern was a long-worrisome circularity in the sociology of science based on the use of bright lines such as $p < 0.05$: "We teach it because it's what we do; we do it because it's what we teach." This concern was brought to the attention of the ASA Board.

The ASA Board was also stimulated by highly visible discussions over the last few years. For example, ScienceNews (Siegfried 2010) wrote: "It's science's dirtiest secret: The 'scientific method' of testing hypotheses by statistical analysis stands on a flimsy foundation." A November 2013, article in Phys.org Science News Wire (2013) cited "numerous deep flaws" in null hypothesis significance testing. A ScienceNews article (Siegfried 2014) on February 7, 2014, said "statistical techniques for testing hypotheses ... have more flaws than Facebook's privacy policies." A week later, statistician and "Simply Statistics" blogger Jeff Leek responded. "The problem is not that people use P -values poorly," Leek wrote, "it is that the vast majority of data analysis is not performed by people properly trained to perform data analysis" (Leek 2014). That same week, statistician and science writer Regina Nuzzo published an article in *Nature* entitled "Scientific Method: Statistical Errors" (Nuzzo 2014). That article is now one of the most highly viewed *Nature* articles, as reported by altmetric.com (<http://www.altmetric.com/details/2115792#score>).

2014) and a statement on risk-limiting post-election audits (American Statistical Association 2010). However, these were truly policy-related statements. The VAM statement addressed a key educational policy issue, acknowledging the complexity of the issues involved, citing limitations of VAMs as effective performance models, and urging that they be developed and interpreted with the involvement of statisticians. The statement on election auditing was also in response to a major but specific policy issue (close elections in 2008), and said that statistically based election audits should become a routine part of election processes.

By contrast, the Board envisioned that the ASA statement on p -values and statistical significance would shed light on an aspect of our field that is too often misunderstood and misused in the broader research community, and, in the process, provides the community a service. The intended audience would be researchers, practitioners, and science writers who are not primarily statisticians. Thus, this statement would be quite different from anything previously attempted.

The Board tasked Wasserstein with assembling a group of experts representing a wide variety of points of view. On behalf of the Board, he reached out to more than two dozen such people, all of whom said they would be happy to be involved. Several expressed doubt about whether agreement could be reached, but those who did said, in effect, that if there was going to be a discussion, they wanted to be involved.

Over the course of many months, group members discussed what format the statement should take, tried to more concretely visualize the audience for the statement, and began to find points of agreement. That turned out to be relatively easy to do, but it was just as easy to find points of intense

t-tests in Python

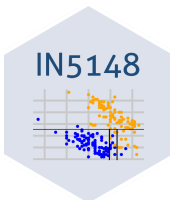
Unfortunately, there is no native function in **scikit-learn** to compute all elements of t-tests for a linear regression model.

The easiest and safest way to do this is to train a model from scratch using the **statsmodels** library in Python.



statsmodels library

- **statsmodels** is a powerful Python library for statistical modeling, data analysis, and hypothesis testing.
- It provides classes and functions for estimating statistical models.
- It is built on top of libraries such as **NumPy**, **SciPy**, and **pandas**
- <https://www.statsmodels.org/stable/index.html>



An issue with **statsmodels** is that we must add the intercept to our predictor matrix using the `add_constant()` function.

```
1 auto_X_train_with_intercept = sm.add_constant(auto_X_train)
```

We fit or train a linear regression model using the `.OLS()` and `.fit()` functions.

```
1 # Create linear regression object in statsmodel
2 regr = sm.OLS(auto_Y_train, auto_X_train_with_intercept)
3
4 # Train the model using the training sets
5 linear_model = regr.fit()
```

We access the information on the t-tests using the `.summary()` function.

```
1 model_summary = linear_model.summary()
2 print(model_summary)
```

OLS Regression Results

```
=====
Dep. Variable:          mpg      R-squared:          0.700
Model:                  OLS      Adj. R-squared:      0.698
Method:                 Least Squares      F-statistic:      453.2
Date:                   Thu, 19 Mar 2026      Prob (F-statistic):  2.43e-102
Time:                   18:09:57      Log-Likelihood:     -1125.4
No. Observations:       392      AIC:                2257.
Df Residuals:           389      BIC:                2269.
Df Model:                2
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025
0.975]					

Table of t-tests

	coef	std err	t	P> t	[0.025	0.975
const	41.0953	1.868	21.999	0.000	37.423	44.768
weight	-0.0073	0.000	-25.966	0.000	-0.008	-0.007
acceleration	0.2617	0.086	3.026	0.003	0.092	0.432

The column **std err** contains the values of the **estimated standard error** ($\sqrt{\hat{v}_{jj}}$) of the estimates $\hat{\beta}_j$. They represent the variability in the estimates.

	coef	std err	t	$P> t $	[0.025	0.975
const	41.0953	1.868	21.999	0.000	37.423	44.768
weight	-0.0073	0.000	-25.966	0.000	-0.008	-0.007
acceleration	0.2617	0.086	3.026	0.003	0.092	0.432

The column **t** contains the values of the **observed statistic t_0** for the hypothesis tests.

	coef	std err	t	$P> t $	[0.025	0.975
const	41.0953	1.868	21.999	0.000	37.423	44.768
weight	-0.0073	0.000	-25.966	0.000	-0.008	-0.007
acceleration	0.2617	0.086	3.026	0.003	0.092	0.432

The column $P>|t|$ contains the **p-values** for the hypothesis tests.

Since the p-value is smaller than $\alpha = 0.05$, we reject H_0 for acceleration and weight. Therefore, **weight and acceleration have a significant association with the mpg of a car.**

	coef	std err	t	$P> t $	[0.025	0.975]
const	41.0953	1.868	21.999	0.000	37.423	44.768
weight	-0.0073	0.000	-25.966	0.000	-0.008	-0.007
acceleration	0.2617	0.086	3.026	0.003	0.092	0.432

The columns [0.025 and 0.975] show the limits of a 95% confidence interval on each true coefficient β_j . This interval *contains* the true parameter value with a confidence of 95%.

Residual analysis

Residuals

The errors of the estimated model are called **residuals**

$$\hat{\epsilon}_i = Y_i - \hat{Y}_i, i = 1, \dots, n.$$

In linear regression, we can validate the adequacy of the model by checking whether the residuals $\hat{\epsilon}_1, \hat{\epsilon}_2, \dots, \hat{\epsilon}_n$ satisfy three (or four) assumptions.

Assumptions

The residuals $\hat{\epsilon}_i$'s must then satisfy the following assumptions:

1. On average, they are close to zero for any value of the predictors X_j .
2. For any value of the predictor X_j , the dispersion or variance is roughly the same.
3. The ϵ_i 's are independent from each other.
4. The $\hat{\epsilon}_i$'s follow normal distribution.

To evaluate the assumption of a linear regression model, we use a **Residual analysis**.

Residual Analysis

To check the validity of these assumptions, we will follow a graphical approach. Specifically, we will construct three informative plots of the residuals.

1. **Residuals vs Fitted Values Plot.** To assess the structure of the model and check for constant variance
2. **Residuals Vs Time Plot.** To check independence.
3. **Normal Quantile-Quantile Plot.** To assess if the residuals follow a normal distribution

Example 2

- This example is inspired by Foster, Stine and Waterman (1997, pages 191–199).
- The data are in the form of the time taken (in minutes) for a production run, Y , and the number of items produced, X , for 20 randomly selected orders as supervised by a manager.
- We wish to develop an equation to model the relationship between the run time (Y) and the run size (X).

Dataset

The dataset is in the file “production.xlsx”.

```
1 production_data = pd.read_excel('production.xlsx')  
2 production_data.head()
```

	Case	RunTime	RunSize
0	1	195	175
1	2	215	189
2	3	243	344
3	4	162	88
4	5	185	114

Linear regression model

$$\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_i$$

where X_i is the run size and $\hat{Y}_i$ is the predicted run time for the i -th observation, $i = 1, \dots, 20$.

We fit the model in **scikit-learn** using the entire data for training.

```
1 # Defining the predictor (X) and the response variable (Y).
2 prod_Y_train = production_data['RunTime']
3 prod_X_train = production_data[['RunSize']]
4
5 # Fitting the simple linear regression model.
6 LRmodelProd = LinearRegression()
7 LRmodelProd.fit(prod_X_train, prod_Y_train)
```

Estimated model

The estimated coefficients are

```
1 print("Intercept:", LRmodel.intercept_)  
2 print("Coefficients:", LRmodel.coef_)
```

So, the fitted model is

$$\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_i = 149.75 + 0.26 X_i,$$

where $\hat{Y}_i$ is the *predicted* or *fitted* value of run time for the i -th observation, $i = 1, \dots, 20$.

The **residuals** of this estimated model are $\hat{\epsilon}_i = Y_i - \hat{Y}_i$.

Calculation of fitted values and residuals

Recall that we can calculate the predicted values and residuals as follows.

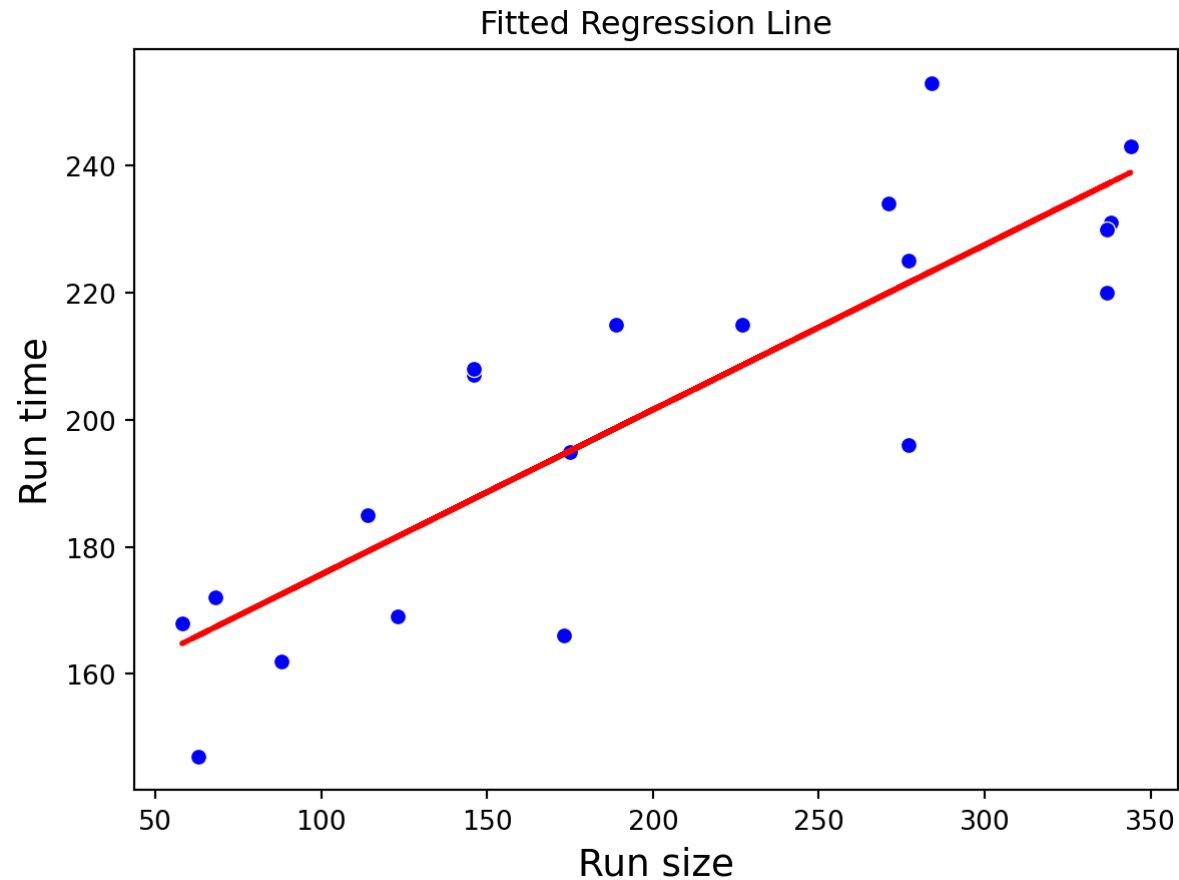
```
1 # Make predictions using the the model
2 prod_Y_pred = LRmodelProd.predict(prod_X_train) + 0*prod_Y_train
3
4 # Calculate residuals.
5 residuals = prod_Y_train - prod_Y_pred
```

The extra `0*prod_Y_train` above allows us to have the correct Python for the predictions from **scikit-learn** functions.

Residuals vs Fitted Values Plot

Intuition behind ...

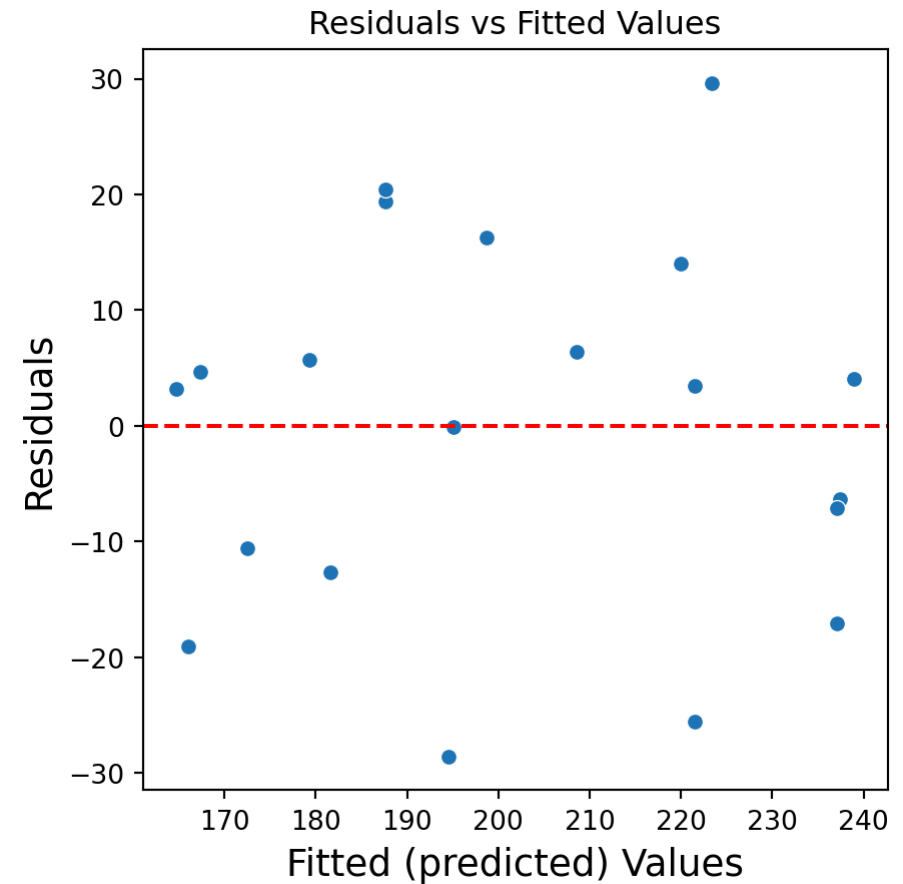
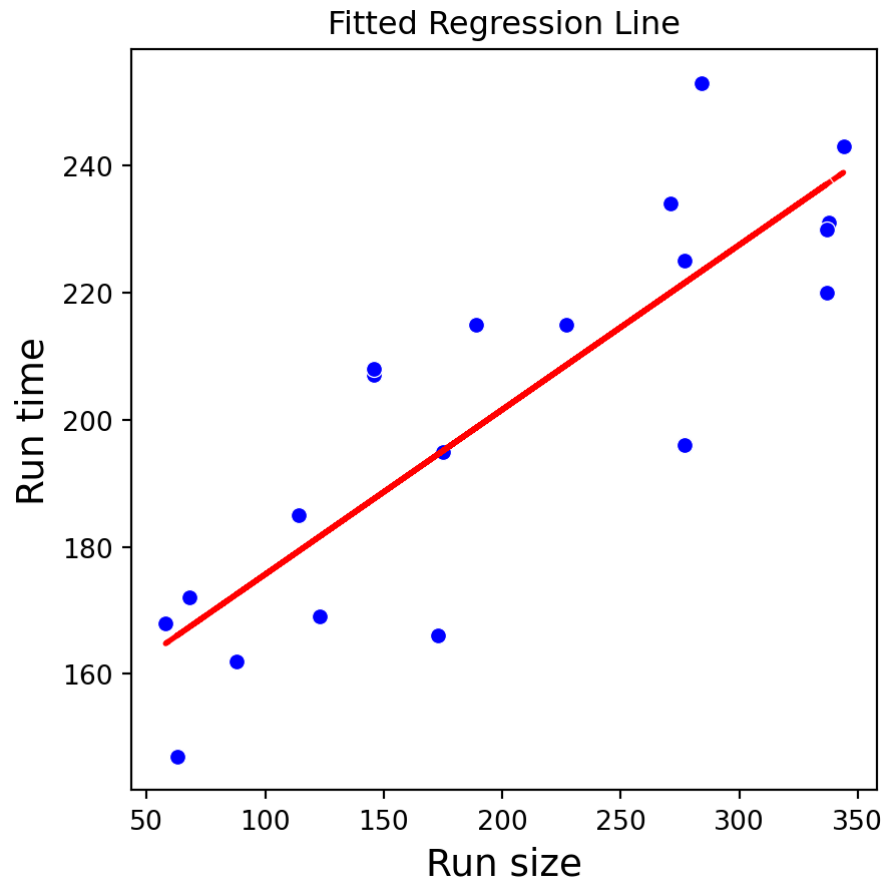
the Residuals versus Fitted Values plot.



Residuals vs Fitted Values

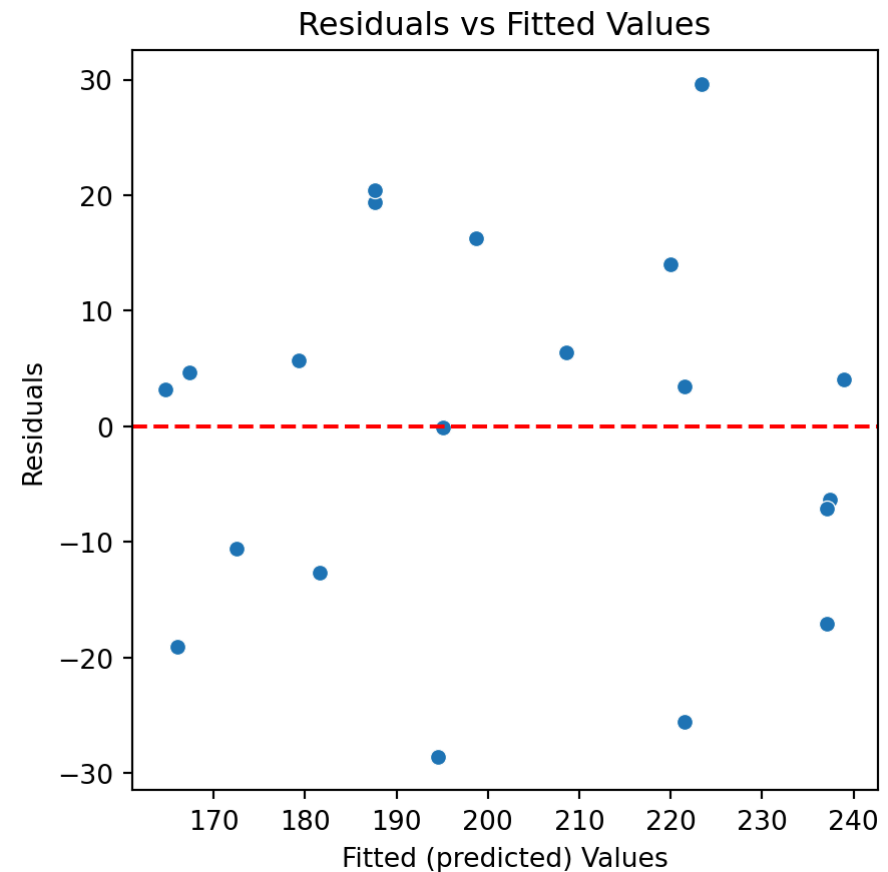
► Code

► Code

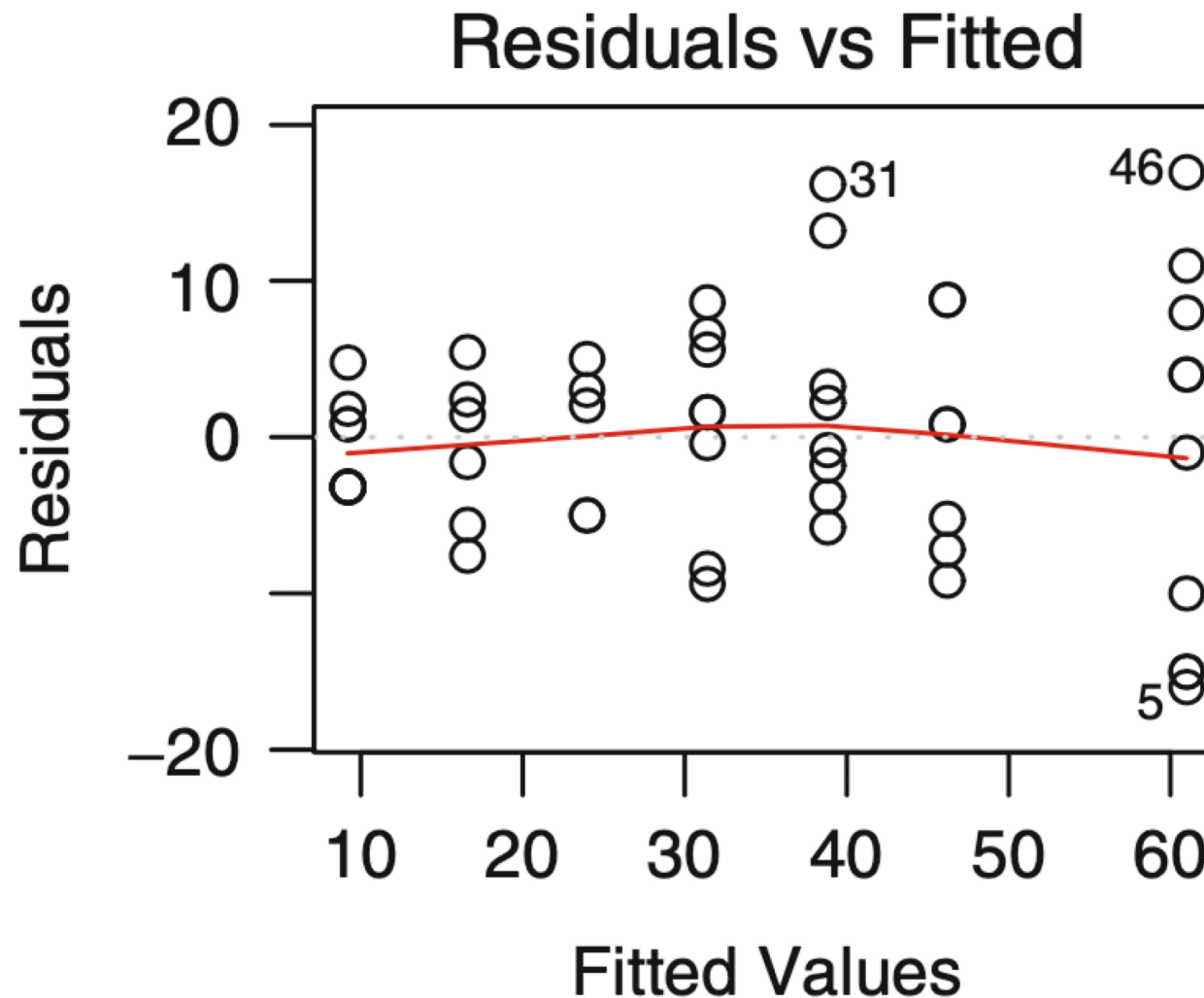


Residuals vs Fitted Values

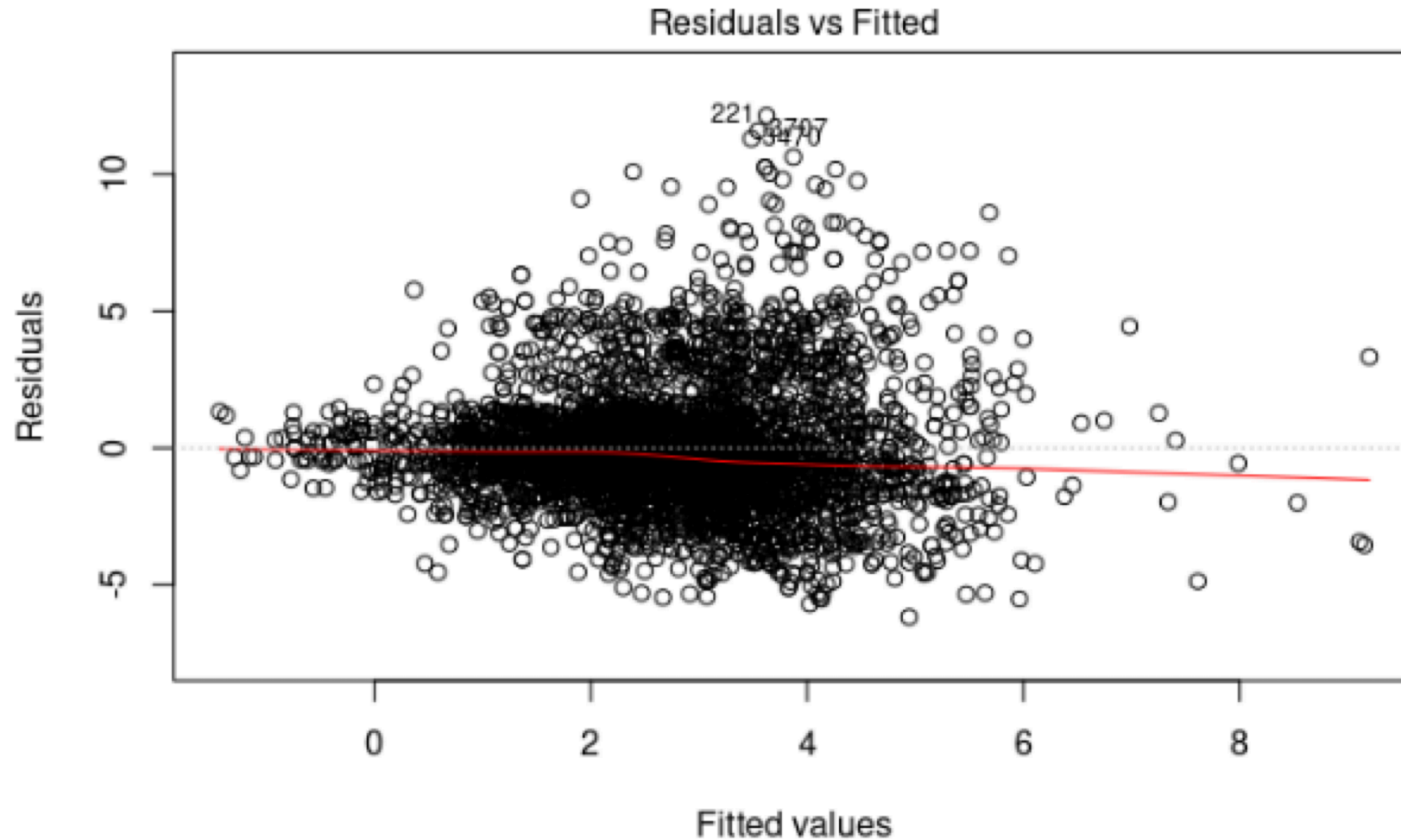
- If there is a trend, the model is misspecified.
- A “funnel” shape indicates that the assumption of constant variance is not met.



Examples of plots that do not support the conclusion of constant variance.



Another example.



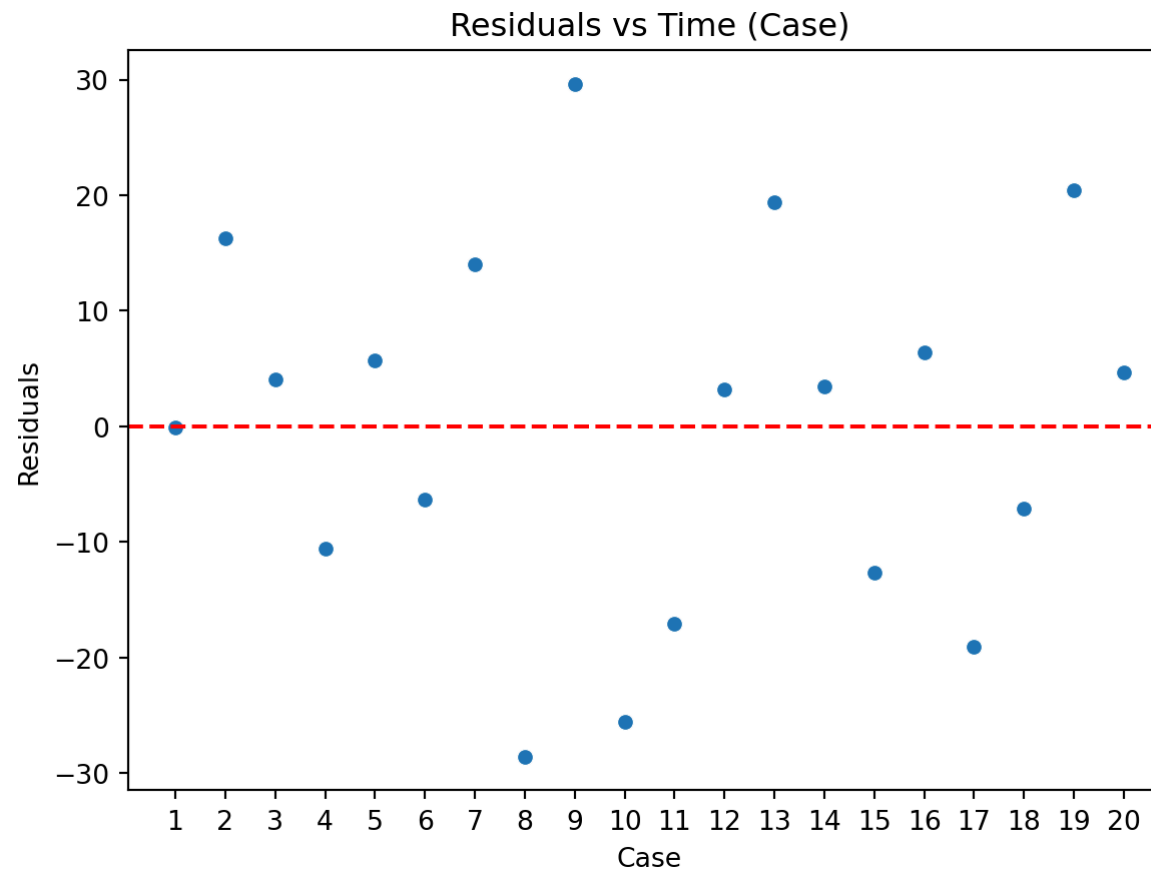
The phenomenon of non-constant variance is called **heteroscedasticity**.

Residuals vs Time Plot

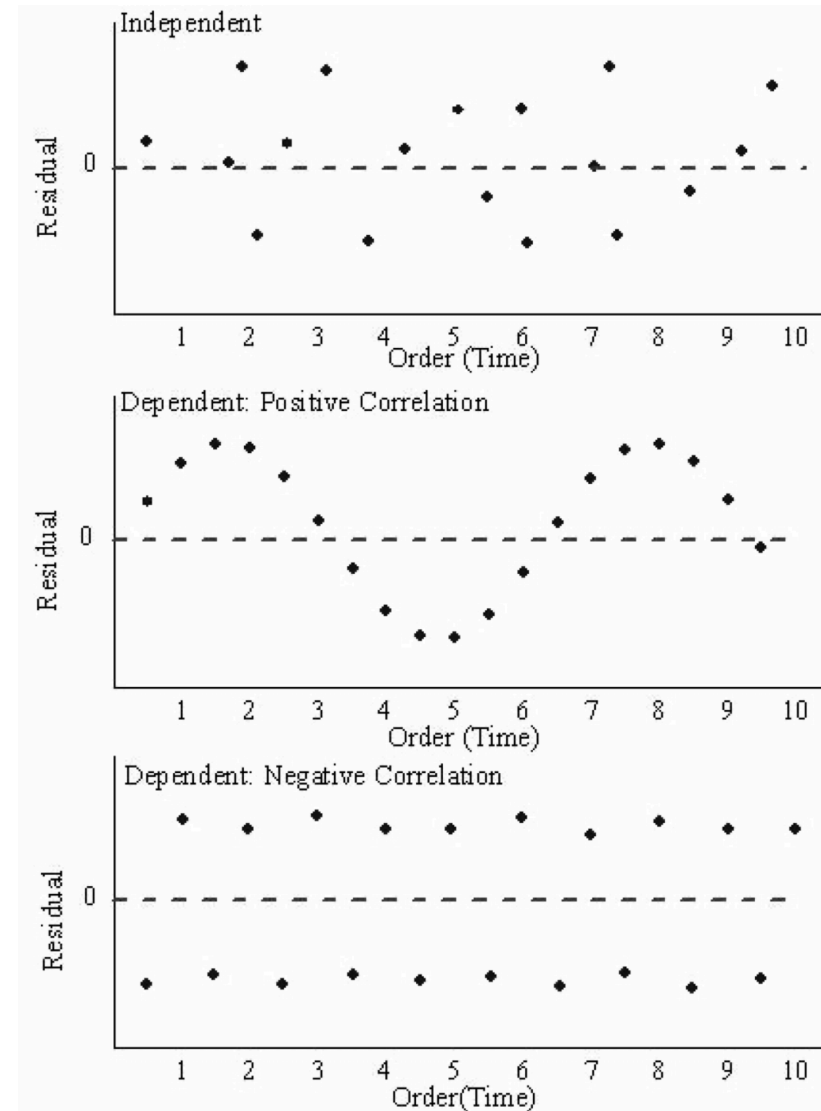
Residuals vs Time Plot

- By “time,” we mean that time the observation was taken or the order in which it was taken. The plot should not show any structure or pattern in the residuals.
- Dependence on time is a common source of lack of independence, but other plots might also detect lack of independence.
- Ideally, we plot the residuals versus each variable of interest we could think of, either included or excluded in the model.
- Assessing the assumption of independence is hard in practice.

► Code



Examples of plots that do and do not support the independence assumption.



Normal Quantile-Quantile Plot

Checking for normality

This assumption is generally checked by looking at the distribution of the residuals.

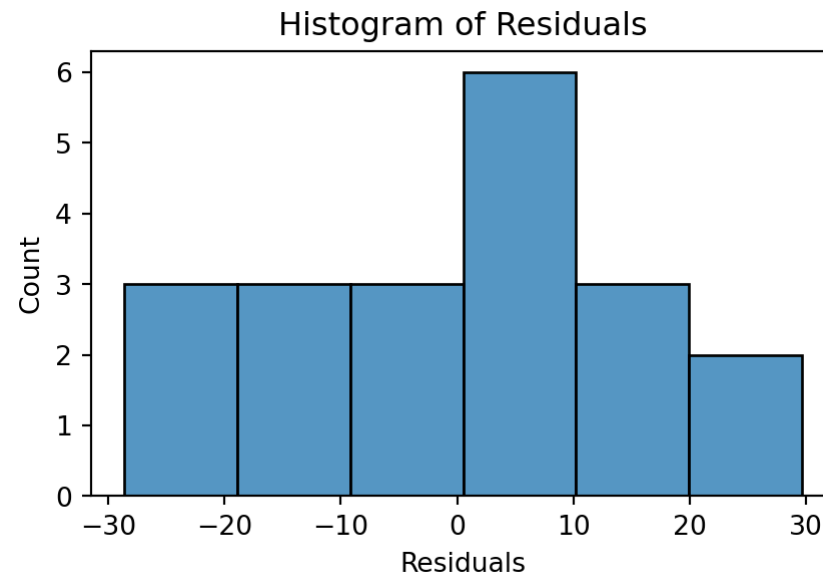
Two plots:

- Histogram.
- Normal Quantile-Quantile Plot (also called normal probability plot).

Histogram

Ideally, the histogram resembles a normal distribution around 0. If the number of observations is small, the histogram may not be an effective visualization.

► Code



Normal Quantile-Quantile (QQ) Plot

A normal QQ plot is helpful for deciding whether a sample was drawn from a distribution that is approximately normal.

1. First, let $\hat{\epsilon}_{[1]}, \hat{\epsilon}_{[2]}, \dots, \hat{\epsilon}_{[n]}$ be the residuals ranked in an increasing order, where $\hat{\epsilon}_{[1]}$ is the minimum and $\hat{\epsilon}_{[n]}$ is the maximum. These points define the **sample** percentiles (or quantiles) of the distribution of the residuals.
2. Next, calculate the **theoretical** percentiles of a (standard) Normal distribution using Python.

- The normal QQ plot displays the (sample) percentiles of the residuals versus the percentiles of a normal distribution.
- If these percentiles agree with each other, then they would approximate a straight line.
- The straight line is usually determined visually, with emphasis on the central values rather than the extremes.
- For a nice explanation, see this [YouTube video](#).

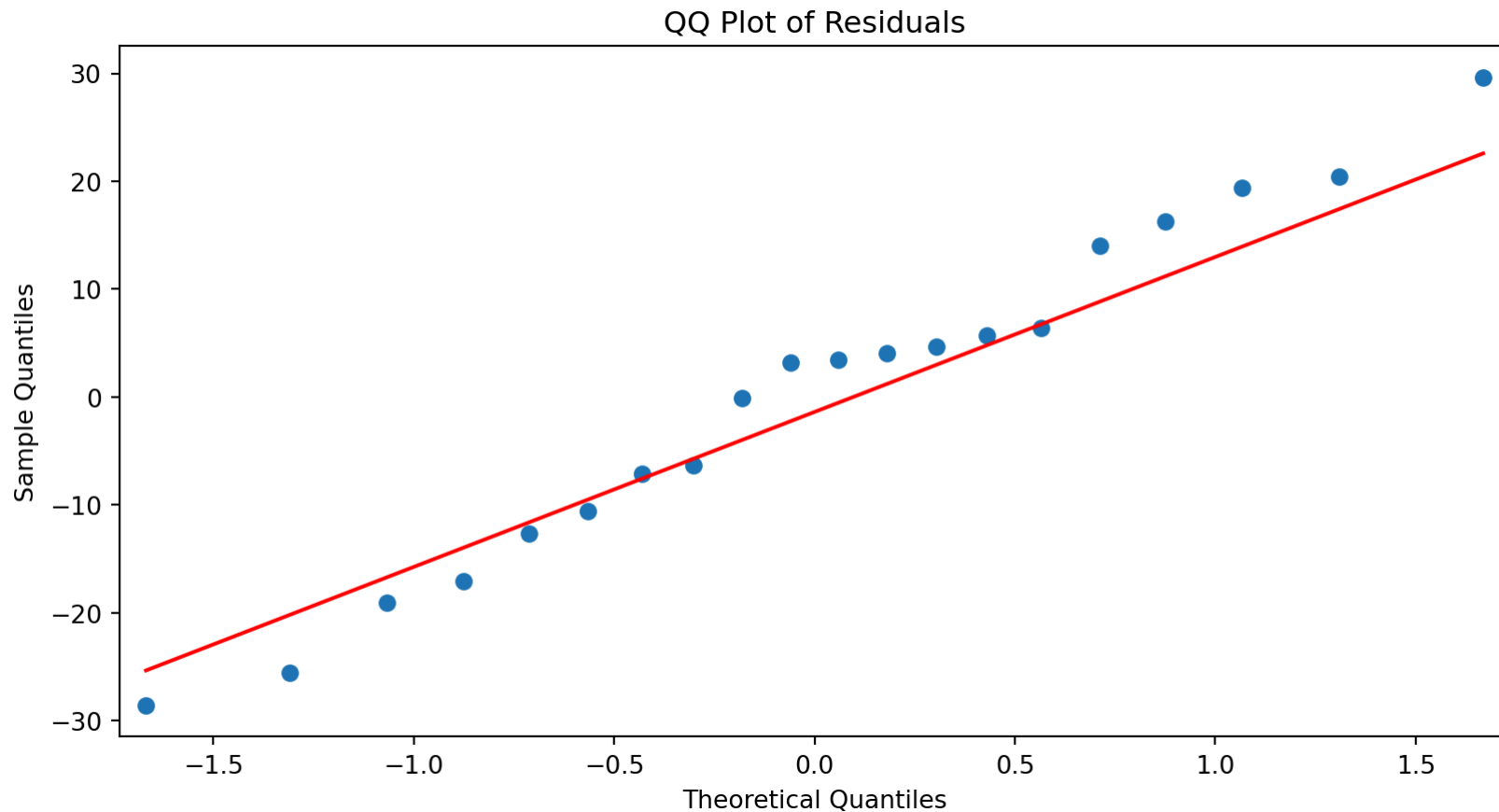
QQ plot in Python

To construct a QQ plot, we use the function `qqplot()` from the **statsmodels** library.

```
1 # QQ plot to assess normality of residuals
2 plt.figure(figsize=(5, 3))
3 sm.qqplot(residuals, line = 'q')
4 plt.title('QQ Plot of Residuals')
5 plt.show()
```

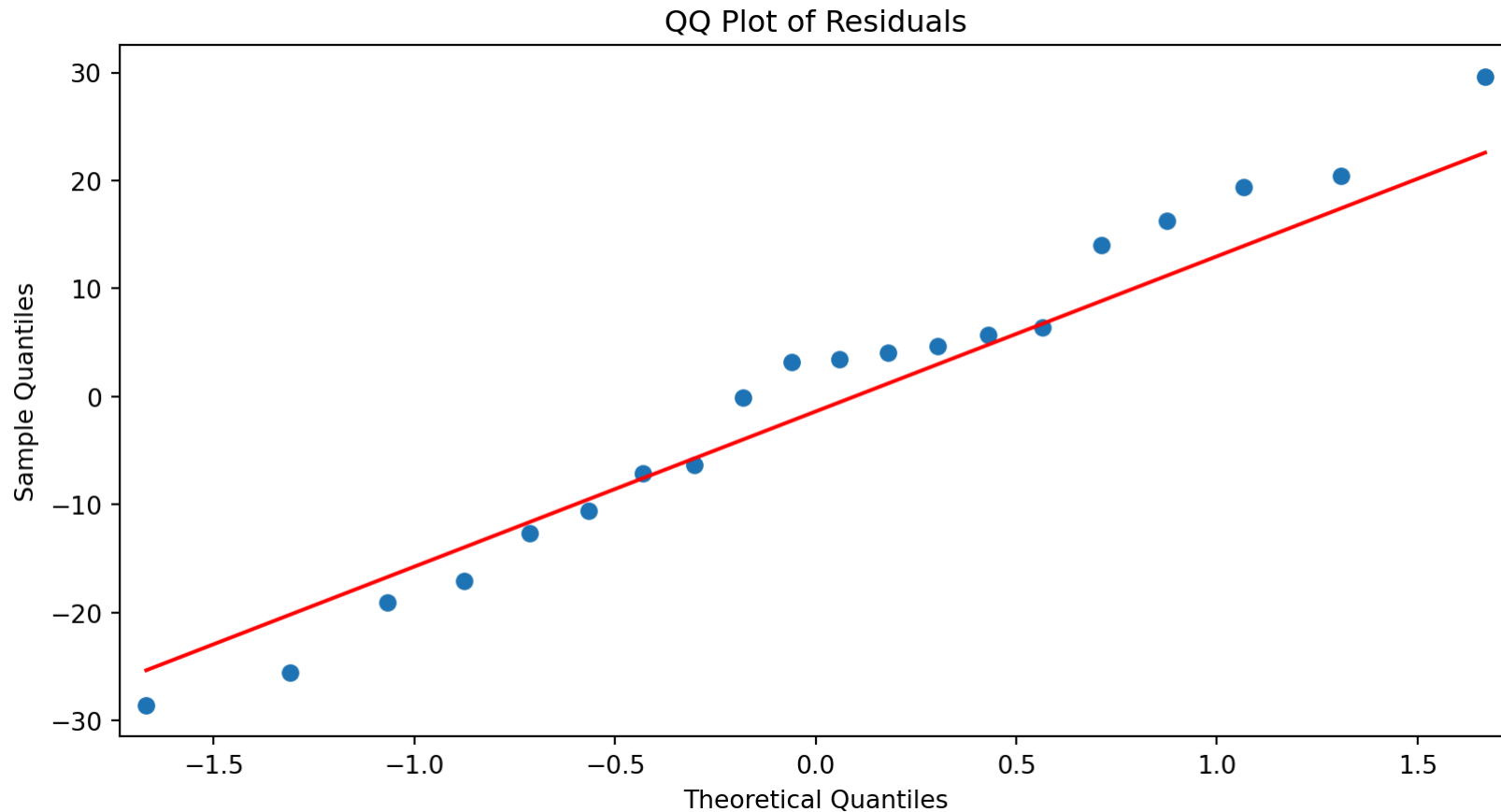
Substantial departures from a straight line indicate that the distribution is not normal.

<Figure size 672x384 with 0 Axes>



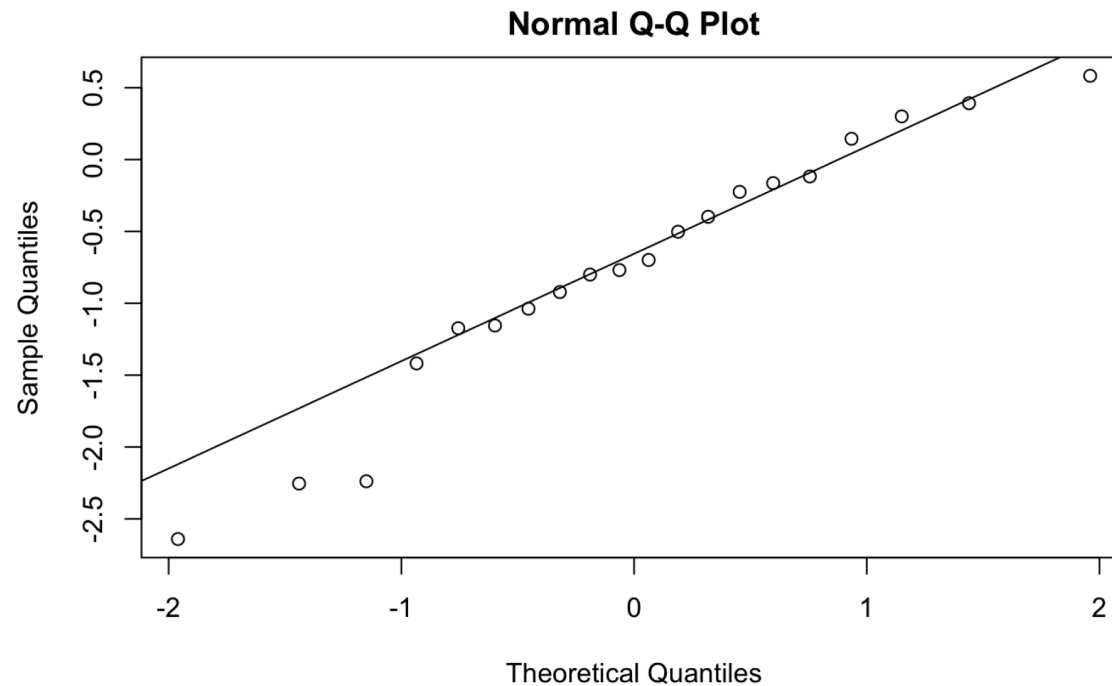
This plot suggests that the residuals are consistent with a Normal curve.

<Figure size 672x384 with 0 Axes>



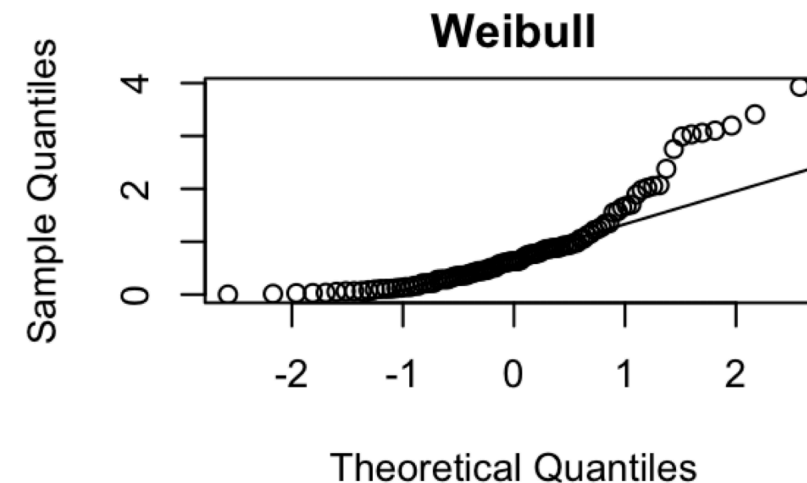
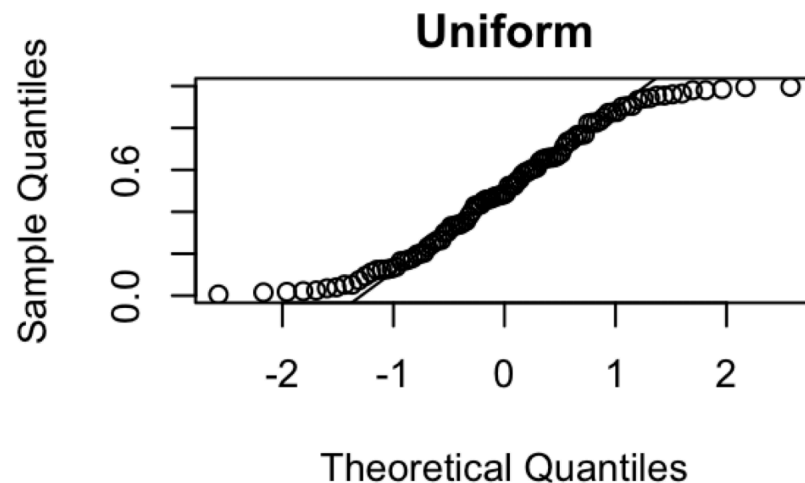
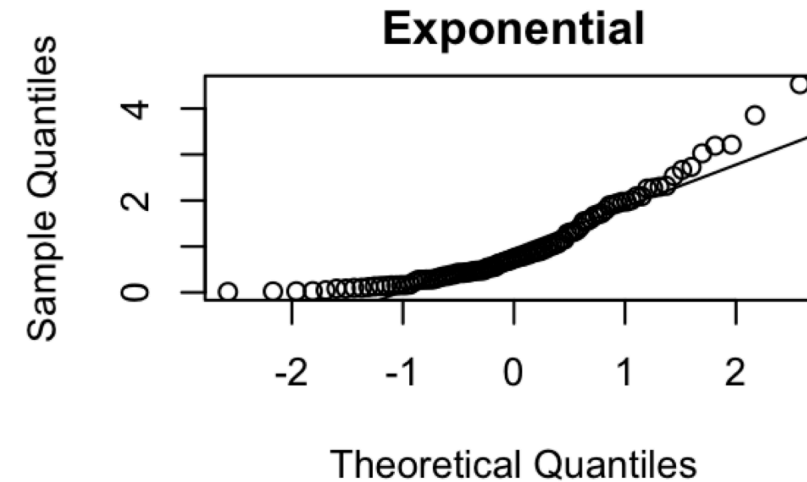
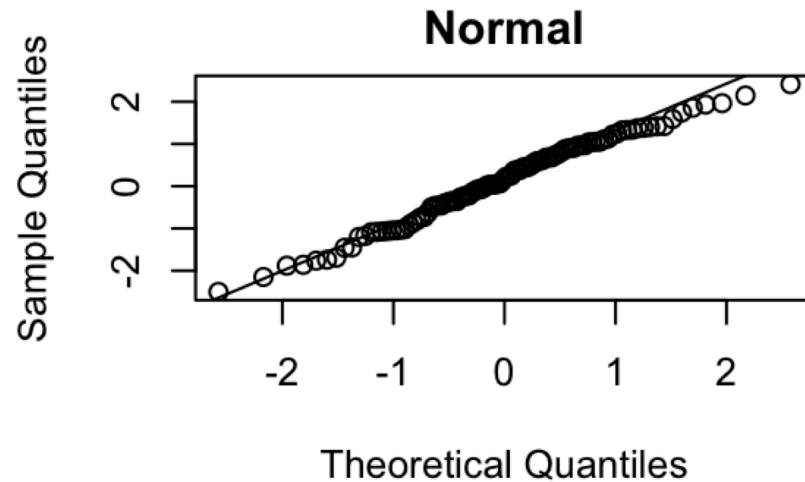
Comments

These data are truly Normally distributed. But note that we still see deviations. These are entirely due to chance.



When n is relatively small, you tend to see deviations, particularly in the tails.

Normal probability plots for data sets following various distributions. 100 observations in each data set.



Consequences of faulty assumptions

1. **If the model structure is incorrect**, the estimated coefficients $\hat{\beta}_j$ will be incorrect and the predictions $\hat{Y}_i$ will be inaccurate.
2. **If the residuals do not follow a normal distribution**, we have two cases:
 - If sample size is large, we still get accurate p-values for the t-tests for the coefficients thanks to the *Central Limit Theorem*.
 - Otherwise, the t-tests and p-values are unreliable.

3. If the residuals do not have constant variance, then the linear model is incorrect and everything falls apart!
4. If the residuals are dependent, then the linear model is incorrect and everything falls apart!

There are methods to *correct* the faulty assumptions, but we will not discuss them here.

Return to main page